

Tarea 1

Las preguntas 1 a 3 requieren un desarrollo computacional usando los códigos Python que se le entregaron, con modificaciones que tendrán que hacer ustedes. Lean bien los enunciados de las tres preguntas ya que están conectados entre sí.

Consideraremos aquí el problema

$$\min_{x \in \mathbb{R}^n} \frac{1}{2} x^T Q x + c^T x$$

donde $Q \in \mathbb{R}^{n \times n}$ es una matriz simétrica y definida positiva y $c \in \mathbb{R}^n$.

En estos problemas, queremos estudiar el comportamiento del Método del Gradiente “clásico” y del Método Simple de Primer Orden, en versión básica y acelerada, aplicado a este problema.

Pregunta 1: (10 pts)

Revise el material sobre la convergencia del Método del Gradiente para el problema de arriba, en los apuntes del curso, sección 2.4.1. Lo mismo está revisado en el video complementario entregado. Como podrá ver, la velocidad de convergencia depende de $\kappa(Q) = \mu_{\max}/\mu_{\min}$, donde $\mu_{\max}$ y $\mu_{\min}$ son el mayor y menor valor propio de la matriz Q , respectivamente. Este número se conoce como el “condicionamiento” de Q y, como se explica en el material, está relacionado al grado de excentricidad de las curvas de nivel de la función cuadrática.

(Nota: Si tienen debilidades en conceptos de normas, valores propios y similares, especialmente normas de matrices, revisen la cápsula de video de repaso inicial que está disponible en el Canvas).

- a) Modifique el código Python del Método del Gradiente, que se entregó en clases, para que sea aplicable a la función de este problema (Ind: esto es más fácil de lo que parece, note qué diferencia hay entre la función de este problema y la usada como ejemplo en clases).

Solución

Hay que cambiar las siguientes variables en la función `subrutina`

```
1 funcion_objetivo = 0.5 * np.dot(np.transpose(x), np.dot(Q, x)) + np.dot(np.transpose(c), x)
2 gradiente = np.dot(Q, x) + c
3 hessiano = Q
```

En la función `funcion_enunciado` hay que cambiar el `return` a

```
1 return (
2     0.5 * np.dot(np.transpose(x), np.dot(Q, x))
3     + np.dot(np.transpose(c), x)
4 ) [0] [0]
```

Además, es necesario modificar la rutina que calcula la función y su gradiente. Esto es fácil considerando que el código original mostrado en clases se aplica a la función

$$f(x) = \frac{1}{2} x^T Q x + c^T x + \alpha(5 - x_n)^4$$

Entonces, basta poner $\alpha = 0$ y tenemos la función de esta tarea. En realidad, es más eficiente dejar el código “limpio” y eliminar toda la parte del término $\alpha(5 - x_n)^4$.

- b) Modifique el código Python del Método del Gradiente o use el código para el Método Simple de Primer Orden para el problema de mínimos cuadrados, para construir una nueva rutina que ejecute el Método Simple de Primer Orden para el problema de esta tarea (note que el código para el problema de mínimos cuadrados es “casi” lo que necesita). Para esto deberá calcular la constante de Lipschitz adecuada, investigue funciones de cálculo numérico de Python que le puedan servir para esto.

Supongamos que vamos a modificar la rutina del Método Simple de Primer Orden. Aquí hay que hacer algunos cambios ya que la función objetivo es para un problema algo diferente, el de mínimos cuadrados:

$$\min_{x \in \mathbb{R}^n} \frac{1}{2} \|Ax - b\|_2^2$$

En nuestro caso, la función objetivo es $f(x) = \frac{1}{2}x^T Qx + c^T x$. Entonces, hay que actualizar el cálculo de esa función objetivo y del gradiente, el cual es:

$$\nabla f(x) = Qx + c$$

También hay que actualizar el cálculo de la constante de Lipschitz, que en este caso es:

$$L = \|Q\|_2 = \mu_{max}$$

donde μ_{max} denota el más grande valor propio de Q .

Adicionalmente, se debe modificar la generación del problema ya que ahora hay que generar una matriz Q definida positiva y un vector c . la rutina `parametros.py`, entregada junto con código del Método del Gradiente, genera lo que necesitamos. Para poder generar la matriz definida positiva existen varios métodos, uno de ellos es usar la función `make_sdp_matrix` de `scikit-learn`.

Así, finalmente el cambio más sencillo de hacer, es modificar en el archivo `steepest.py` el cálculo del λ y reemplazarlo por

```
1 lambda_ = 1 / np.linalg.norm(Q, 2)
```

- c) Modifique también el código Python del Método Simple de Primer Orden Acelerado de Nesterov para el problema de mínimos cuadrados, para adaptarlo al problema de esta tarea.

Solución

Para esto, modificamos primero los parámetros de la función, para que quede como `FOMLS_acc(Q, c, x0, L, epsilon, numero_iteraciones)` y posterior a eso modificamos las líneas del gradiente y el valor, para que queden como

```
1 gradiente = np.dot(Q, yax) + c
2 valor = 0.5 * np.dot(xk.T, np.dot(Q, xk)) + np.dot(c.T, xk)
```

- d) Complemente los tres códigos para que calculen, también, el condicionamiento de la matriz Q Revise todos los códigos entregados, en alguna parte eso ya se hace).

En esta pregunta, debe incluir las transcripciones de los códigos producidos en su respuesta a la tarea.

Solución

Basta con agregar esta línea a cada uno de los códigos

```
1 np.linalg.cond(Q)
```

Pregunta 2: (10 pts)

Ahora vamos a ejecutar el método del Gradiente sobre un conjunto de prueba de problemas generados en forma aleatoria. Para esto, observe el programa `main.py`, que es el que controla la ejecución de los métodos. Modifique la semilla del generador de números aleatorios usando la fecha de nacimiento de algún integrante

del grupo, en el formato DDMMAAAA. Ahora, configure el código sucesivamente para generar y correr el Método del Gradiente para problemas con $n = 10, 50, 100, 150, 200, 250, 500, 750, 1000, 2000, 4000$, hasta llegar a un error de $\epsilon = 0,0001$. Si es necesario, ajuste el límite de iteraciones también.

Para cada ejecución del Método del Gradiente en los problemas registre: condicionamiento de la matriz Q generada, número de iteraciones, tiempo de ejecución (que es desplegado al final por los códigos de clases), y cualquier otro antecedente del proceso de convergencia.

Puede ocurrir que tenga problemas de convergencia, especialmente para las instancias más grandes. En ese caso, ajuste los parámetros de la función `fminbnd`. También puede, si quiere, reemplazar el uso de la función `fminbnd` por una fórmula explícita para el linesearch. Esta fórmula, de hecho, existe en el caso de querer minimizar una función cuadrática (la puede encontrar en las referencias del curso).

Solución

Al ejecutar el método del gradiente se deberían evidenciar dos comportamientos al menos. Estos son en que a medida que aumenta el n cada iteración se va haciendo más pesada, y que mientras va aumentando el número de condicionamiento, también aumenta considerablemente el número de iteraciones para converger.

Un ejemplo de resultado de ejecución es el siguiente en donde se tiene registrada solo la última iteración y se puso un tope de 50 iteraciones

Cuadro 1: Ejemplo resultados de iteraciones y tiempos de ejecución método gradiente						
Iteración	Valor Obj	Norma	Lambda	Cond(Q)	n	segundos
50	-3.896951	0.268657	0.11687594	118.760542	10	0.009001
50	-29.223232	2.815058	0.02008217	20108.844693	50	0.015509
50	-33.078471	4.421483	0.01010305	27369.162761	100	0.317143
50	-85.368717	5.787311	0.00668188	8311.502063	150	1.284885
50	-69.611014	11.354363	0.00504504	93264.857043	200	1.839453
50	-151.716824	12.136007	0.00401033	339101.606240	250	3.015384
50	-164.639718	20.476563	0.00200553	549153.826985	500	10.496093
50	-221.000503	31.873843	0.00133643	1075851.210774	750	23.745745
50	-398.969203	46.242608	0.00100107	1968771.849191	1000	39.844326
50	-572.594162	77.592825	0.00050046	3044288.465370	2000	105.640083
50	-1849.874824	158.772112	0.00025004	19161862.943175	4000	427.029385

Nota importante: El valor objetivo siempre debería ir decreciendo, si de una iteración a otra el valor aumenta, es porque hay algo mal implementado.

Pregunta 3: (10 pts)

Ahora, ejecute el Método Simple de Primer Orden y también el Método Acelerado de Nesterov, sobre exactamente los mismos datos de entrada que en la Pregunta 2 (si usa la misma semilla y los mismos n , eso es lo que ocurrirá).

Para cada ejecución de ambos métodos en los problemas registre: condicionamiento de la matriz Q generada (ya lo tiene de la pregunta anterior, en realidad), número de iteraciones, tiempo de ejecución, y cualquier otro antecedente del proceso de convergencia. Puede ser que para n grande la convergencia se haga muy lenta, tendrán que tomar alguna decisión sobre cómo abordar esto o, simplemente, interrumpir la ejecución antes.

Compare las corridas de los tres métodos. Haga eso en forma explícita con gráficos que muestren la convergencia y también gráficos de algún indicador de convergencia y condicionamiento de Q . ¿Puede decir algo sobre el efecto del condicionamiento de Q en uno u otro? ¿Puede decir algo sobre las diferencias de tiempo

de ejecución? Analice qué factores del problema pueden estar influyendo en las diferencias.

Solución

Para el método de simple de primer orden, con un tope de iteraciones de 50, se podría obtener la siguiente tabla (es solo a modo de ejemplo)

Cuadro 2: Resultados del método de primer orden para distintos tamaños de n .

Iteración	Valor Obj	Norma	Lambda	Cond(Q)	n	segundos
50	-1.732591	0.195105	0.094949	118.760542	10	0.001497
50	6.405505	4.295376	0.019797	20108.844693	50	0.004000
50	31.255775	8.708089	0.009949	27369.162761	100	0.206674
50	62.532960	9.032490	0.006644	8311.502063	150	1.052279
50	103.836779	15.467534	0.004988	93264.857043	200	1.524839
50	128.457616	12.840746	0.003992	339101.606240	250	2.501707
50	360.472880	26.146566	0.001998	549153.826985	500	9.050016
50	604.497607	35.559383	0.001332	1075851.210774	750	20.473690
50	741.971209	35.025571	0.001000	1968771.849191	1000	37.909874
50	1763.733857	60.879609	0.000500	3044288.465370	2000	111.549462
50	2775.766950	60.530329	0.000250	19161862.943175	4000	461.252356

En el caso del método acelerado de Nesterov se tiene la siguiente tabla de ejemplo

Cuadro 3: Resultados del método acelerado de Nesterov para distintas dimensiones n .

Iteración	Valor Obj	Norma	Cond(Q)	n	Segundos
314	-1.917917	0.000099	118.760542	10	0.000000
1000	-45.383615	0.008127	20108.844693	50	0.016520
1000	-66.861525	0.034606	27369.162761	100	0.248185
1000	-95.030259	0.020093	8311.502063	150	0.265209
1000	-109.231556	0.067731	93264.857043	200	0.291839
1000	-172.537267	0.177556	339101.606240	250	0.306558
1000	-198.064147	0.157854	549153.826985	500	0.605079
1000	-301.118538	0.437326	1075851.210774	750	0.986140
1000	-476.982121	0.757787	1968771.849191	1000	1.523166
1000	-749.171011	1.321010	3044288.465370	2000	4.229734
1000	-1996.500880	2.386804	19161862.943175	4000	21.770802

Comentarios:

1. Los resultados de las tablas adjuntos para cada uno de los métodos tienen como fin ejemplificar un posible comportamiento y posible forma de abordar el análisis. Los resultados obtenidos por los grupos pueden ser radicalmente distintas, ya que va a depender de la semilla utilizada, pero las conclusiones en todos deberían ser similares.
2. Para el método de primer orden deberían observar que cada iteración toma menos tiempo, pero toma más iteraciones en converger, lo anterior tiene que ver por el uso del λ predefinido de antemano.
3. Para el método de Nesterov se debería observar que no se ve tan afectado ni por las dimensiones ni por el valor de Q , como lo es en los otros algoritmos. En un inicio parte más o menos un descenso lento, y después avanza rápidamente para luego avanzar de a poquito de nuevo, porque queda bien cerca del óptimo.

Pregunta 4: (12 pts)

Considere ahora la función que se mostró en clases, para ilustrar los métodos de Gradiente y Newton,

$$f(x) = \frac{1}{2}x^T Qx + c^T x + \alpha(5 - x_n)^4$$

donde $Q \in \mathbb{R}^{n \times n}$ es simétrica y definida positiva, $c \in \mathbb{R}^n$, α es constante.

a) (4 pts) En este caso, $\nabla^2 f(x)$ no es una matriz constante, depende de x . Argumente que $\nabla f(x)$ NO tiene propiedad de Lipschitz globalmente en $\mathbb{R}^n$.

Solución

Nos piden demostrar que no se cumple

$$\begin{aligned} \|\nabla f(x) - \nabla f(y)\| &\leq L\|x - y\| \quad / : \|x - y\| \\ \frac{\|\nabla f(x) - \nabla f(y)\|}{\|x - y\|} &\leq L \quad / x \rightarrow y \\ \nabla^2 f(x) &\leq L \end{aligned}$$

Considerando $g(x) = \frac{1}{2}x^T Qx + c^T x$ y $h(x) = \alpha(5 - x_n)^4$. Se tiene que

$$\nabla g(x) = Qx + c \qquad \qquad \nabla^2 g(x) = Q$$

Por otra parte

$$\frac{\partial h(x)}{\partial x_i} = \begin{cases} -4\alpha(5 - x_n)^3 & i = n \\ 0 & e.o.c \end{cases}$$

$$\frac{\partial^2 h(x)}{\partial x_j \partial x_i} = \begin{cases} 12\alpha(5 - x_n)^2 & i = j = n \\ 0 & e.o.c \end{cases}$$

Por tanto, como hay una parte de $\nabla^2 f(x) = \nabla^2 g(x) + \nabla^2 h(x)$, que depende de x , la función $\nabla f(x)$ podría no cumplir la propiedad de Lipschitz, como, de hecho, ocurre aquí.

En efecto, tomemos x^* y otro punto y . Debería cumplirse que

$$\|\nabla f(x^*) - \nabla f(y)\| \leq L\|x^* - y\|$$

Pero $\nabla f(x^*) = 0$, así que debería cumplirse que

$$\|\nabla f(y)\|_2 \leq L\|x^* - y\|_2$$

(hacemos el argumento con la norma 2) Pero, dada la expresión para $\nabla f(x)$, tenemos que:

$$\|\nabla f(y)\|_2 = \|Qy + c + \nabla h(y)\|_2$$

Podemos ver que la expresión para $\nabla h(y)$ agrega un término de la forma $12\alpha(5 - y_n)$ a la norma. Basta elegir cualquier vector en donde la coordenada y_n crezca de valor en forma arbitraria y estaremos sumando un término cúbico, que crece muy rápido comparado con el crecimiento de Qy , que es lineal. Por otro lado, $\|x^* - y\|$ sólo crece en forma lineal en y_n . Se concluye, entonces, que no puede existir un L fijo que haga cumplir la desigualdad

$$\|\nabla f(y)\|_2 = \|Qy + c + \nabla h(y)\|_2$$

y no se cumple que ∇f tenga propiedad de Lipschitz.

- b) (4 pts) Por otro lado, supongamos que “sabemos” que las iteraciones del método siempre estarán contenidas en una región $D = \{x : \|x\|_2 \leq R\}$, donde R es un valor fijo. Argumente que, en ese caso, Sí es posible definir un valor de constante de Lipschitz para $\nabla f(x)$ que sea válida en D .

Solución

Es sabido que se puede estimar L al que $\max \|\nabla^2 f(x)\|_2 = L$. Como se nos indica que x está contenido en una región D , dadas las características del algoritmo de descenso, se sabe que las siguientes iteraciones van a estar contenidas en D , por tanto, se puede estimar una constante L de tal manera que

$$L = \max_{s.a. \|x\|_2 \leq R} \|\nabla^2 f(x)\|_2$$

Como el problema es compacto (cerrado y acotado), el problema tiene solución óptima, y por tanto, existe una constante L para la región D . Usando las expresiones anteriores se puede, de hecho, calcular una cota superior a esto, que también sirve de constante L . Tenemos que

$$\|\nabla^2 f(x)\|_2 = \|Q + B(x)\|_2$$

donde $B(x)$ es la matriz que es idénticamente cero, excepto en el elemento (n, n) , el que vale $12\alpha(5 - x_n)^2$. Entonces,

$$\|\nabla^2 f(x)\|_2 \leq \|Q\|_2 + \|B(x)\|_2 = \|Q\|_2 + |12\alpha(5 - x_n)^2|$$

En esto último, hemos usado la propiedad de que $\|A + B\| \leq \|A\| + \|B\|$ y el hecho que la norma de $B(x)$ es simplemente el valor de su único elemento diagonal no nulo.

Entonces, si $\|x\|_2 \leq R$, tenemos que $|x_n| \leq R$, lo que nos permite acotar:

$$\|\nabla^2 f(x)\|_2 \leq \|Q\|_2 + 12\alpha|(5 - x_n)|^2 \leq \|Q\|_2 + 12\alpha|(5 + R)|^2$$

y ahí tenemos la cota y posible valor para L . Recordemos que $\|Q\|_2$ es un valor bien definido, corresponde al mayor valor propio de la matriz Q .

- c) (4 pts) Esto de tener un valor de R con la propiedad indicada en la parte b), puede ser algo complejo. Sin embargo, le pedimos que argumente que si α es pequeño, podemos cometer un error grande al estimar R y aún así tener una eventual constante de Lipschitz “razonable”.

Solución

Si α es lo suficientemente pequeño, esto quiere decir que a medida que $\alpha \rightarrow 0$, se cumple que $12\alpha|(5+R)|^2 \approx 0$. Por tanto, puede ser considerado despreciable y, efectivamente, un error en la estimación de R tendrá muy poco impacto en lo que se suma a $\|Q\|_2$.

Pregunta 5 (8 pts):

- a) (4 pts) Muchos computadores tienen, por ejemplo, un procesador Intel, o de otro fabricante. Elijan el computador de uno de los integrantes del grupo e identifiquen el tipo de procesador. Busquen en Internet la información (no olvide poner la cita) que le permita determinar cuantos “flop” por segundo puede realizar ese procesador. Con esa información en mano, construyan una tabla que indique cuanto tiempo se requiere para una iteración del Método de Newton, que asumiremos (como explicamos en clases) necesita ejecutar n^3 flops, una iteración de un método Quasi-Newton, que asumiremos requiere n^2 flops, y una iteración del método de Gradiente, que asumiremos requiere n flops. En la tabla ponga su estimación del tiempo que se requiere para realizar una iteración de cada uno de esos métodos. Haga esta tabla para los siguientes valores de n , $n = 50, 250, 500, 1,000, 5,000, 20,000, 150,000, 2,000,000, 10,000,000, 100,000,000$.

(Un problema de optimización con millones de variables se puede dar en Machine Learning, reconstrucción de imágenes u otras áreas). Desde luego estos cálculos están ignorando otras operaciones que también requieren tiempo (estimar los gradientes y Hessianos, el “linesearch”) y el hecho que el procesador no necesariamente estará exclusivamente dedicado a su trabajo, pero se obtendrá, al menos, una idea aproximada de los tiempos necesarios y su dependencia en n para cada método.

Solución

Una posible tabla, para un i7-1065G7 con $8,3 \cdot 10^{10}$ flops (el computador del profesor) es la siguiente

Tiempo requerido en segundos	50	250	500	1,000	5,000	20,000	150,000	2,000,000	10,000,000	100,000,000
n^3	1.5E-06	0.00019	0.00151	0.01205	1.506024	96.385542	40662.65	96385542	1.2048E+10	1.2048E+13
n^2	3E-08	7.5E-07	3E-06	1.2E-05	0.000301	0.0048193	0.271084	48.192771	1204.81928	120481.928
n	6E-10	3E-09	6E-09	1.2E-08	6.02E-08	2.41E-07	1.81E-06	2.41E-05	0.00012048	0.00120482
Tiempo en años	50	250	500	1,000	5,000	20,000	150,000	2,000,000	10,000,000	100,000,000
n^3	4.8E-14	6E-12	4.8E-11	3.8E-10	4.78E-08	3.056E-06	0.001289	3.0563655	382.045687	382045.687
n^2	9.6E-16	2.4E-14	9.6E-14	3.8E-13	9.55E-12	1.528E-10	8.6E-09	1.528E-06	3.8205E-05	0.00382046
n	1.9E-17	9.6E-17	1.9E-16	3.8E-16	1.91E-15	7.641E-15	5.73E-14	7.641E-13	3.8205E-12	3.8205E-11

Figura 1: Tabla tiempo operación computador i7-1065G7

- b) (4 pts) Ahora, busque información respecto al supercomputador más rápido existente en este momento (suele cambiar año a año, así que busque información actualizada al día de hoy). Una vez identificado, fíjese en la información de su capacidad de cálculo, en “flops” por segundo. En su solución indique de qué supercomputador se trata y cuáles son sus características (no olvide indicar las fuentes de su información), y repita los cálculos de la parte a) para este caso y compare. ¿Qué opina respecto al balance entre capacidad de cálculo (según número de núcleos) y la posibilidad de resolver un problema? (Si les interesa saber hacia dónde piensan algunos que podría llegar la capacidad de computación, en algún futuro, busquen en Internet lo que se dice sobre un “Matrioshka Brain”).

Solución

A día de hoy, el supercomputador más potente es El Capitan - HPE Cray EX255a, AMD 4th Gen EPYC 24C 1.8GHz, AMD Instinct MI300A, Slingshot-11, Toss . El máximo teórico de flops, son 2,746.38 PFlop/s. En cuanto al número de núcleos vs la capacidad de resolver un problema, no necesariamente tener más cores implica tener mejor rendimiento. Hay que tener en cuenta que tan paralelizable es el código y que escalabilidad presenta (fuerte o débil). También hay otros elementos asociados que hay que tener en cuenta, como la latencia y el ancho de banda con el que opera el supercomputador, si es que experimenta mucha latencia o tiene poca banda ancha, la capacidad de procesamiento y de recibir información por parte del computador se va a ver fuertemente limitada. Por último, otro factor a considerar es que tan bien está implementado un cierto algoritmo, si por ejemplo tiene muchos caché misses, aun teniendo los procesadores más rápidos, las ineficiencias en la programación pueden dar a que no se le saque todo el provecho a ese gran número de cores.

En cuanto a la capacidad de cómputo de El Capitán, se resume en la siguiente tabla

Tiempo requerido en segundos	50	250	500	1,000	5,000	20,000	150,000	2,000,000	10,000,000	100,000,000
n^3	4.63E-14	5.79E-12	4.63E-11	3.70E-10	4.63E-08	2.96E-06	1.25E-03	2.96E+00	3.70E+02	3.70E+05
n^2	9.26E-16	2.31E-14	9.26E-14	3.70E-13	9.26E-12	1.48E-10	8.33E-09	1.48E-06	3.70E-05	3.70E-03
n	1.85E-17	9.26E-17	1.85E-16	3.70E-16	1.85E-15	7.41E-15	5.56E-14	7.41E-13	3.70E-12	3.70E-11
Tiempo en años	50	250	500	1,000	5,000	20,000	150,000	2,000,000	10,000,000	100,000,000
n^3	1.47E-21	1.84E-19	1.47E-18	1.17E-17	1.47E-15	9.40E-14	3.96E-11	9.40E-08	1.17E-05	1.17E-02
n^2	2.94E-23	7.34E-22	2.94E-21	1.17E-20	2.94E-19	4.70E-18	2.64E-16	4.70E-14	1.17E-12	1.17E-10
n	5.87E-25	2.94E-24	5.87E-24	1.17E-23	5.87E-23	2.35E-22	1.76E-21	2.35E-20	1.17E-19	1.17E-18

Figura 2: Tabla tiempo operación El Capitán

Pregunta 6 (10 pts):

Considere la siguiente función en n variables:

$$f(x) = \sum_{j=1}^{n-1} |x_j - x_{j+1}| + \|Ax - b\|_2^2$$

donde $A \in \mathbb{R}^{m \times n}$ y $b \in \mathbb{R}^n$. El primer término de esta función es lo que se conoce como una “medida de variación total”.

a) (2 pts) ¿Es convexa esta función? Justifique todo con precisión.

Solución

Por un lado, se tiene que por lo visto en clase, el término $\|Ax - b\|_2^2$ es convexo, ya que corresponde a un término cuadrático. Por otra parte, el término $\sum_{j=1}^{n-1} |x_j - x_{j+1}|$ se puede reescribir como $\sum_{j=1}^{n-1} \max\{x_j - x_{j+1}, -(x_j - x_{j+1})\}$. Como el máximo de funciones convexas es convexo, se tiene por tanto que esa parte de la función es convexo. Así, como $f(x)$ corresponde a una suma de funciones convexas, $f(x)$ es convexa.

b) (6 pts) Determine $\partial f(x)$ en un punto cualesquiera del dominio de f . Para abordar esto, le va a ser útil la siguiente “regla de la cadena” para funciones no diferenciables: sea f una función convexa, si $p \in \partial f(x)$ y $g(x) = f(Bx)$, donde B es alguna matriz de dimensiones adecuadas, entonces $B^T p \in \partial g(x)$. Tendrá que encontrar explícitamente la matriz B adecuada.

Solución

Sea $g(x) = \sum_{j=1}^{n-1} |x_j - x_{j+1}|$ y $h(x) = \|Ax - b\|_2^2$, donde $f(x) = g(x) + h(x)$. Se cumple así que, $\partial f(x) = \partial g(x) + \partial h(x)$.

$$\begin{aligned} \partial h(x) &= \partial \|Ax - b\|_2^2 = \nabla \|Ax - b\|_2^2 \\ &= 2A^\top (Ax - b) \end{aligned}$$

Ahora analizamos $g(x)$ y para esto notemos que se puede escribir $g(x) = \|Bx\|_1$, donde B es una matriz de $n \times n$ dada de la siguiente forma:

$$B = \begin{bmatrix} 1 & -1 & 0 & \cdots & \cdots & 0 & 0 \\ 0 & 1 & -1 & \cdots & \cdots & 0 & 0 \\ \vdots & \vdots & & & & \vdots & \vdots \\ 0 & & & & & 1 & -1 \\ 0 & 0 & & \cdots & \cdots & 0 & 0 \end{bmatrix}$$

Tenemos, entonces, siguiendo la indicación del enunciado, que

$$\partial g(x) = \partial(\|Bx\|_1) = B^T h(Bx)$$

donde $h(u) \in \partial(\|u\|_1)$

Por otro lado, conocemos muy bien los subgradiientes de la función $\|u\|_1$ ya que se desarrolló en clases. Combinando todo tenemos que $p(x) \in \partial g(x)$ es de la forma:

$$p(x)_j = \begin{cases} d_1(x) & j = 1 \\ d_j(x) - d_{j-1}(x) & j = 2, \dots, n-1 \\ -d_{n-1}(x) & j = n \end{cases}$$

donde

$$d_j(x) = \begin{cases} 1 & \text{si } x_j - x_{j+1} > 0 \\ [-1, 1] & \text{si } x_j - x_{j+1} = 0 \\ -1 & \text{si } x_j - x_{j+1} < 0 \end{cases}$$

c) (2 pts) Determine el esfuerzo computacional, en función de n y m , en el que se incurre para determinar un elemento de $\partial f(x)$, para un x dado en el dominio de f .

Solución

Tenemos que analizar el esfuerzo computacional de los dos sumandos. Para el subgradiente de g podemos ver que el esfuerzo es muy simple, sólo requiere analizar los casos y hacer sumas o restas (no hay que multiplicar matrices y vectores, B la usamos por conveniencia para poder obtener la expresión). Podemos, entonces, considerar este esfuerzo $O(n)$.

Para el segundo término, $2A^\top(Ax - b)$, tenemos que este es lo mismo que $A^T Ax - A^T b$. Tanto $A^T A$ como $A^T b$ se precaculan al inicio de los algoritmos, así que ese esfuerzo no entra en el cálculo de esfuerzo por iteración. Sólo tenemos la multiplicación de una matriz de $m \times n$ por un vector y eso es $O(mn)$. En consecuencia, el esfuerzo total es $O(mn)$.

Pregunta 7 (10 pts):

Considere el siguiente problema de optimización:

$$\begin{aligned} \min \quad & \left\{ \max_{i=1, \dots, m} \{ \alpha_i^T x - b_i \} + \sum_{j=1}^{n-1} (x_j - x_{j+1})^2 \right\} \\ \text{s.a.} \quad & \sum_{j=1}^n x_j \leq 1 \\ & x_j \geq 0, j = 1, \dots, n \end{aligned}$$

Donde $\alpha_i \in \mathbb{R}^n$, $i = 1, \dots, m$, y los b_i son escalares. Esta función es convexa.

a) (6 pts) Determine $\partial f(x)$ en un punto cualesquiera del dominio del problema (para el primer término de la función objetivo, le puede ser útil graficar un caso en una variable, para entender mejor esa función “máx”).

Solución

Sea

$$f(x) = \left\{ \max_{i=1, \dots, m} \{ \alpha_i^T x - b_i \} + \sum_{j=1}^{n-1} (x_j - x_{j+1})^2 \right\}$$

donde se define además $h(x) = \max_{i=1, \dots, m} \{ \alpha_i^T x - b_i \}$ y $g(x) = \sum_{j=1}^{n-1} (x_j - x_{j+1})^2$. Se cumple que $\partial f(x) = \partial h(x) + \partial g(x)$. Así se tiene que

$$\begin{aligned} \partial g(x) &= \partial \sum_{j=1}^{n-1} (x_j - x_{j+1})^2 \\ &= \sum_{j=1}^{n-1} \partial (x_j - x_{j+1})^2 \end{aligned}$$

Como es diferenciable de la manera típica, finalmente es sacar la derivada de esta función. Para esto se hace teniendo cuidado con los índices quedando así

$$\begin{aligned} \frac{\partial g}{\partial x_j} &= 2(x_j - x_{j+1}) - 2(x_{j-1} - x_j) \quad \forall j = 2, \dots, n-1 \\ \frac{\partial g}{\partial x_1} &= 2(x_1 - x_2) \\ \frac{\partial g}{\partial x_n} &= -2(x_{n-1} - x_n) \end{aligned}$$

Ahora

$$\frac{\partial h(x)}{\partial x_j} = \begin{cases} \alpha_{jj'} & \text{si se cumple que } \alpha_j^T x - b_j > \alpha_i^T x - b_i \quad \forall i = 1, \dots, m : i \neq j \\ \sum_{j'=1}^m \lambda_j \alpha_{jj'} & \text{donde } \sum \lambda_j = 1 \quad \text{si se cumple que } \alpha_j^T x - b_j > \alpha_i^T x - b_i \quad \forall i = 1, \dots, m : i \neq j, \forall j \in J \end{cases}$$

en donde J hace referencia al conjunto de índices que alcanzan el máximo. La expresión diferencia el caso en que haya “empates” en el máximo pero corresponde a lo que se obtiene del máximo de funciones, ver el ejemplo 3.4.2 en los apuntes de clases, en donde se desarrolla el cálculo del subgradiente de este tipo de funciones.

b) (4 pts) Queremos resolver este problema usando el Método de Subgradiente. Como el problema tiene algunas restricciones, se debe usar una versión proyectada del método. Describa en detalle todos los pasos del algoritmo, indicando, en particular, la forma de abordar la etapa de proyección.

El problema de proyección que se necesita acá no es de solución inmediata. Si bien parece relacionado con proyectar sobre $\{x : \|x\|_1 \leq 1\}$, que es muy directo, en este caso no es tan así ya que el punto a proyectar puede estar en cualquier parte y la proyección debe cumplir $x \geq 0$.

Hay referencias sobre este problema en la web. El siguiente desarrollo está basado en <https://math.stackexchange.com/questions/2402504/orthogonal-projection-onto-the-unit-simplex>. Otra referencia posible es <https://doi.org/10.48550/arXiv.1101.6081>.

El problema de proyectar un punto y sobre las restricciones especificadas es:

$$\begin{aligned} \text{mín} \quad & \frac{1}{2} \|x - y\|_2^2 \\ \text{s.a.} \quad & \sum_{j=1}^n x_j = 1 \\ & x_j \geq 0, j = 1, \dots, n \end{aligned}$$

Este es un problema de optimización para el que se puede definir el siguiente Lagrangiano:

$$L(x, \mu) = \frac{1}{2} \|x - y\|_2^2 + \mu \left(\sum_{j=1}^n x_j - 1 \right)$$

Entonces, resolvemos el problema $\min_{x \geq 0} L(x, \mu)$, que es

$$\min_{x \geq 0} \sum_{j=1}^n ((x_j - y_j)^2 + \mu x_j) - \mu$$

La solución óptima se obtiene, primero derivando e igualando a cero pero ignorando la restricción $x \geq 0$. Esto da que

$$x_j - y_j + \mu = 0, j = 1, \dots, n$$

y se deduce que el óptimo de todo el problema es

$$x_j^* = \max\{0, (y_j - \mu)\}$$

Ahora, hay que encontrar μ y para eso usamos la restricción. Debe cumplirse que

$$\sum_{j=1}^n x_j^* = \sum_{j=1}^n \max\{0, (y_j - \mu)\} = 1$$

Acá todo está definido y es fácil ver que la función $h(\mu) = \sum_{j=1}^n \max\{0, (y_j - \mu)\}$ es lineal por tramos, su valor va cambiando según como se va comparando μ con los valores y_j . Entonces, encontrar el valor μ que hace que $h(\mu) = 1$ se puede hacer con un proceso de búsqueda. Esto es, comenzar con μ grande (que hace que $h(\mu) = 0$ y después viendo los valores sucesivamente inspeccionando $\mu = y_j$, para los valores y_j en orden descendiente. Eso permite determinar el rango más estrecho entre dos valores y_k y y_{k+1} tales que $h(y_{k+1}) < 1$ y $h(y_k) > 1$. Una vez hecho esto, se despeja el valor de μ en el intervalo que hace la suma exactamente igual a 1. Con eso, queda completamente definida la proyección y esto es lo que se usaría para proyectar el avance del subgradiente. Finalmente, dado que se tiene el problema proyectado, simplemente habría que ejecutar el siguiente algoritmo

Algoritmo simple de primer orden proyectado
Data: $x^0 \in \mathbb{R}^n, k = 0, d^0 = -\nabla f(x^0)$. while <i>Criterio de parada no se satisface</i> do: $x^{k+1} = \Pi_D(x^k + \lambda^k d^k)$ $d^{k+1} = -\nabla f(x^{k+1})$ $k = k + 1$ end

Figura 3: Método primer orden proyectado